Informe de Implementación: Arquitectura CI/CD Desacoplada

Proyecto: Modernización de flujos de emisión para TechMaster LTDA

Preparado por: Bryan Painemilla, Ingeniero DevOps

Institución: Duoc UC

Fecha: 17 de abril de 2026

1. Resumen Ejecutivo

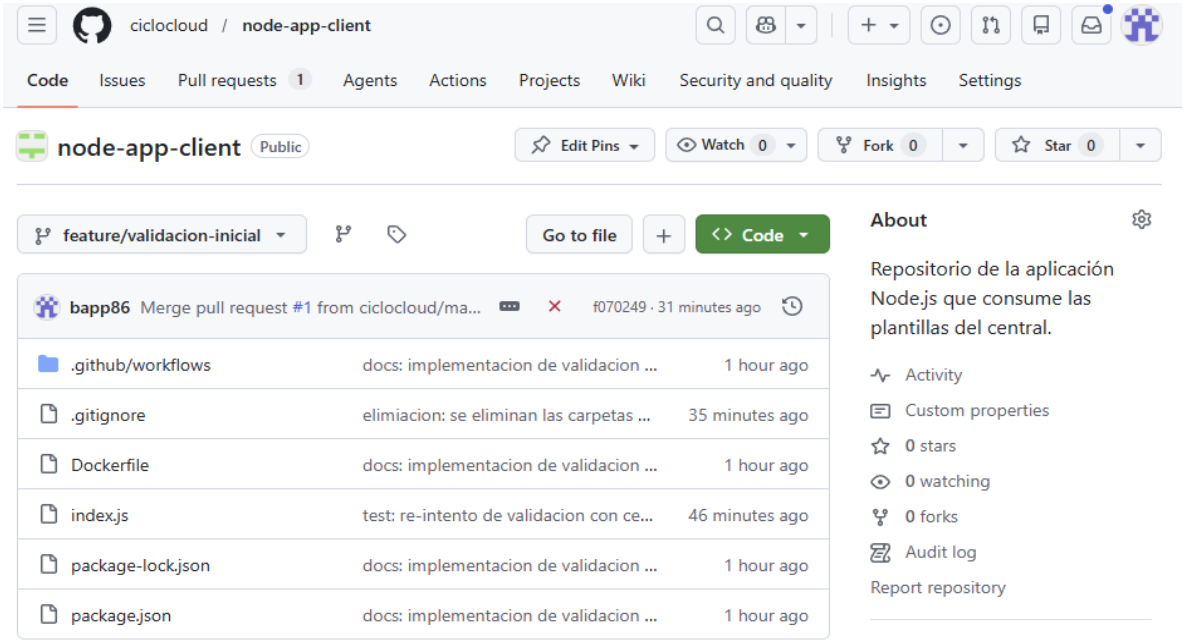
La presente consultoría técnica detalla la transformación de los procesos de integración y despliegue continuo (CI/CD) para *TechMaster LTDA*. El objetivo primordial fue mitigar el riesgo de despliegues fallidos y optimizar el uso de recursos mediante una arquitectura desacoplada.

La solución implementada separa la validación constante del código de la publicación oficial de imágenes de contenedor, garantizando que solo el software que cumple con el 100% de las pruebas unitarias y de construcción sea elegible para su distribución en el registro oficial.

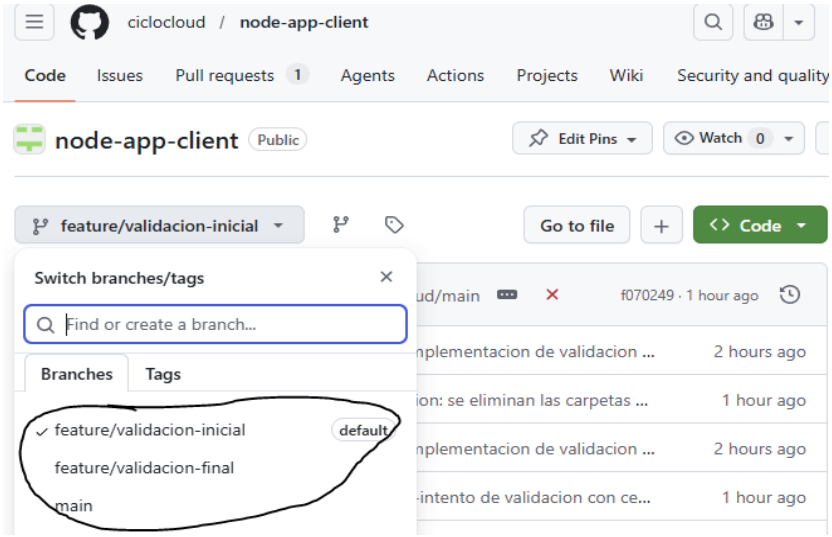
2. Arquitectura de Repositorios (Gobernanza Centralizada)

Se ha establecido un modelo de gobernanza basado en dos pilares fundamentales que aseguran la mantenibilidad a largo plazo:

- **Repositorio Central (central-templates):** Actúa como el motor lógico de la organización. Alberga plantillas YAML reutilizables que estandarizan el ciclo de vida de las aplicaciones Node.js, incluyendo la gestión de caché y capas de seguridad.
- **Repositorio Cliente (node-app-client):** Dedicado exclusivamente al desarrollo de la lógica de negocio. Este repositorio consume la lógica centralizada, reduciendo la deuda técnica y permitiendo una configuración de pipelines minimalista en el lado del cliente.

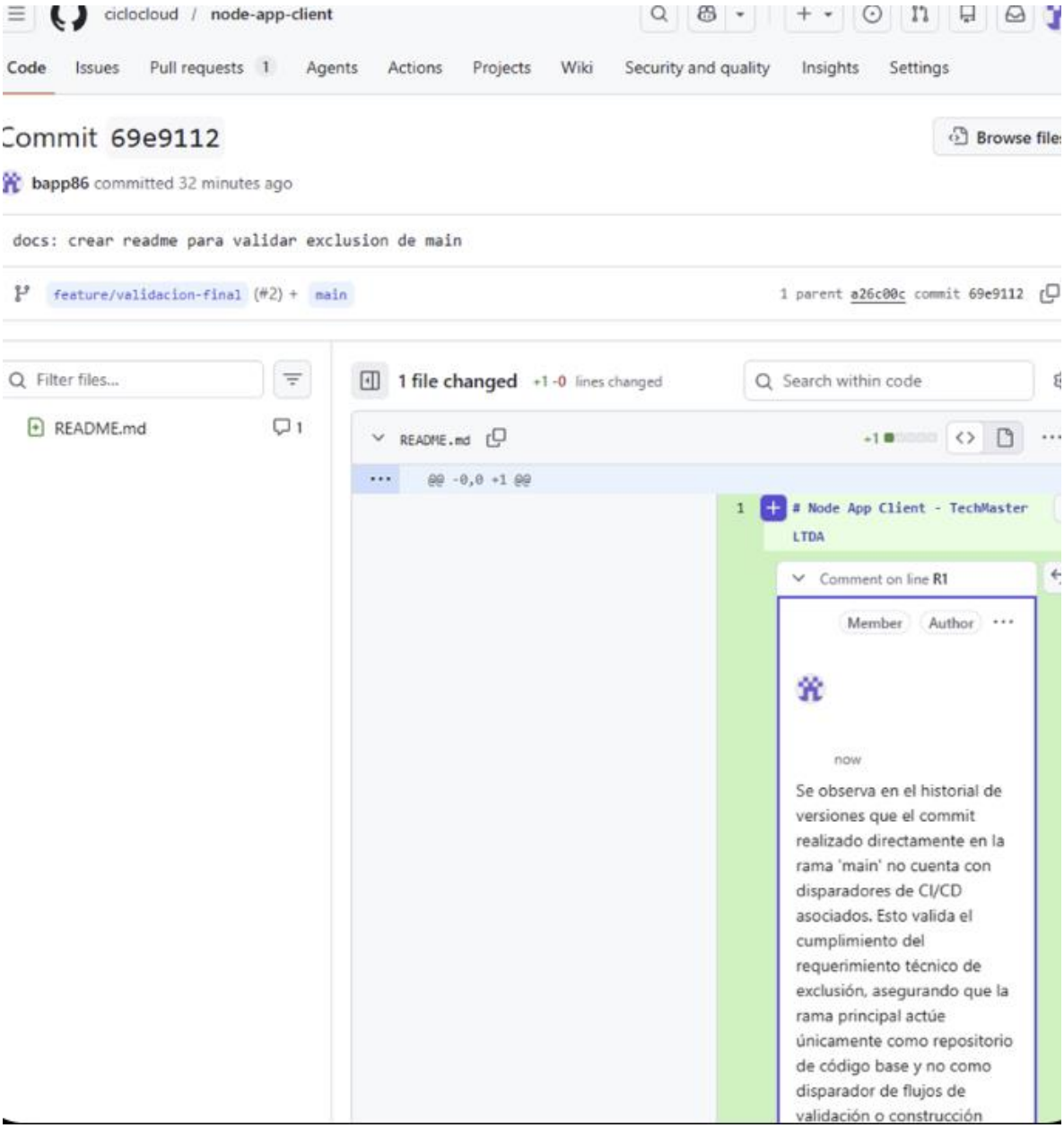


Se observa la correcta exclusión de artefactos locales, delegando la construcción al entorno de CI



Se ha implementado una estrategia de branching basada en Feature Branches, permitiendo el aislamiento total de los cambios antes de su integración. En la evidencia se observa la convivencia de la rama 'main' (fuente de verdad), junto con ramas de desarrollo específico (feature/validacion-inicial y final). Esta segregación garantiza que el flujo de trabajo de CI/CD valide cada funcionalidad de forma independiente, evitando la contaminación del código base durante la fase de experimentación y troubleshooting

3. Control de Ejecución y Estrategia de Disparadores (Triggers)



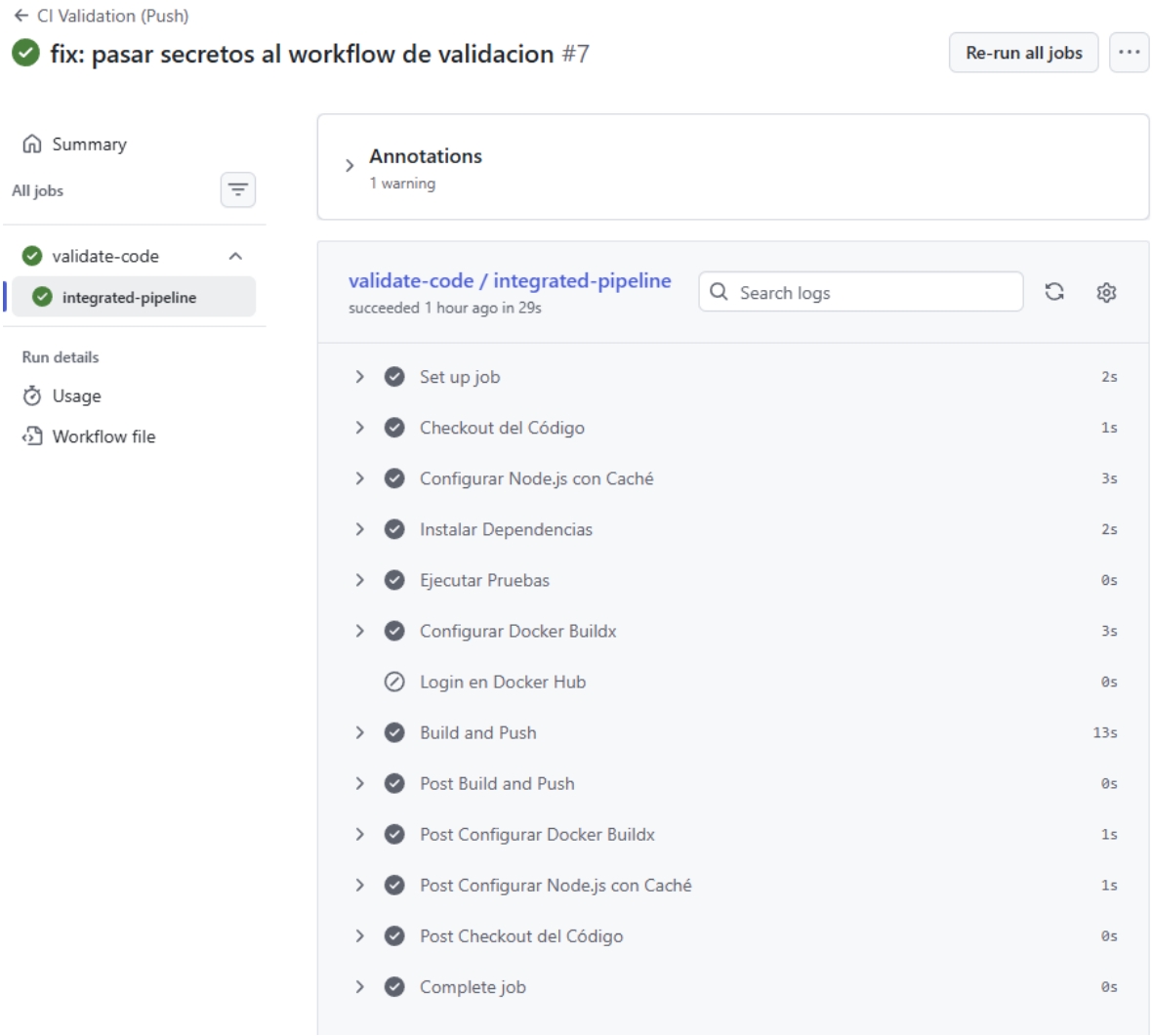
Como parte de la estrategia de Branching, se ha configurado la rama 'main' como una rama de solo almacenamiento de código estable. En la evidencia superior se observa que, tras realizar un commit directo en dicha rama, el sistema de GitHub Actions no inicia ninguna ejecución. Esto garantiza que la infraestructura de TechMaster LTDA no consuma recursos de cómputo innecesarios en versiones que no han pasado por el flujo de validación previo en ramas feature.

El éxito de la arquitectura reside en la gestión inteligente de eventos, cumpliendo con la exigencia de separar la validación del release oficial:

3.1. Validación Desacoplada (CI)

Activada por el evento push en ramas de trabajo (feature/*). Este flujo ejecuta el "Quality Gate":

1. **Instalación Limpia:** Uso de npm ci para garantizar consistencia.
2. **Pruebas Unitarias:** Validación de la lógica de negocio.
3. **Construcción de Imagen:** Verificación de la integridad del Dockerfile



En la imagen se aprecia que el paso "Login en Docker Hub" fue omitido intencionalmente, validando el aislamiento del entorno de pruebas.

3.2. Lanzamiento Oficial (CD)

Activado exclusivamente por **Git Tags** (v*). Este es el único punto del sistema autorizado para realizar la publicación en Docker Hub, utilizando el tag de versión como etiqueta de la imagen.

```
bpainemilla@bpainemillavm001: ~/node-app-client
bpainemilla@bpainemillavm001:~/node-app-client$ git tag
v1.0.0
v1.1.0
v1.2.0
bpainemilla@bpainemillavm001:~/node-app-client$
```

Se evidencia el historial de versiones donde la creación de un tag dispara el despliegue final a producción.

4. Decisiones Técnicas y Optimización de Rendimiento

Para alcanzar el estándar de "Alto Rendimiento" solicitado por TechMaster LTDA, se integraron las siguientes optimizaciones:

- **Gestión de Caché Avanzada:** Se implementó cache-from: type=gha en la construcción de Docker. Esto permite que las capas de la imagen que no han cambiado sean reutilizadas entre ejecuciones, reduciendo los tiempos de construcción de minutos a segundos.
- **Seguridad de Credenciales:** La autenticación con Docker Hub se gestiona mediante *Access Tokens* y *GitHub Secrets*, eliminando la presencia de credenciales en texto plano.
- **Tratamiento de Errores (Troubleshooting):** Durante la implementación se detectaron fallos de formato en etiquetas de imagen, los cuales fueron resueltos centralizando la lógica de etiquetado en la plantilla reutilizable.

← CD Release (Tag)

test: re-intento de validacion con central activo #1

Re-run all jobs | Latest #2 | ···

Summary

All jobs

release-image

integrated-pipeline

Run details

Usage

Workflow file

Annotations
1 warning

release-image / integrated-pipeline
succeeded 1 hour ago in 42s

Search logs

Set up job		3s
Checkout del Código		1s
Configurar Node.js con Caché		3s
Instalar Dependencias		2s
Ejecutar Pruebas		0s
Configurar Docker Buildx		4s
Login en Docker Hub		1s
Build and Push		22s

```
1   ▶ Run docker/build-push-action@v5
12  ▶ GitHub Actions runtime token ACs
15  ▶ Docker info
106 ▶ Proxy configuration
108 ▶ Buildx version
111 ▶ Builder info
164 /usr/bin/docker buildx build --cache-from type=gha --cache-to type=gha,mode=max --
    iidfile /home/runner/work/_temp/docker-actions-toolkit-ABp4N0/build-iidfile-
    8da5253b69.txt --attest type=provenance,mode=max,builder-
    id=https://github.com/ciclocloud/node-app-client/actions/runs/24593688133 --tag
```

release-image / integrated-pipeline

succeeded 1 hour ago in 42s

Search logs

Build and Push 22s

```
#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 DONE 0.0s

#6 [1/5] FROM docker.io/library/node:20-slim@sha256:f93745c153377ee2fbbdd6e24efcd03cd2e86d6ab1d8aa9916a3790c40313a55
#6 resolve docker.io/library/node:20-slim@sha256:f93745c153377ee2fbbdd6e24efcd03cd2e86d6ab1d8aa9916a3790c40313a55 done
#6 DONE 0.0s

#7 importing cache manifest from gha:7889243390998944451
#7 DONE 0.2s

#6 [1/5] FROM docker.io/library/node:20-slim@sha256:f93745c153377ee2fbbdd6e24efcd03cd2e86d6ab1d8aa9916a3790c40313a55
#6 ...

#5 [internal] load build context
#5 transferring context: 2.99MB 0.1s done
#5 DONE 0.2s

#6 [1/5] FROM docker.io/library/node:20-slim@sha256:f93745c153377ee2fbbdd6e24efcd03cd2e86d6ab1d8aa9916a3790c40313a55
#6 sha256:c229a5665a7712c6ffed00bc6c69aa9c0863da6c57230a2a49f67b65ad46cc60 448B / 448B 0.2s done
#6 sha256:9b97559caf99ea6450224af9cae9da7d2a54713725ed842cff9b4e47dfa6f3a2 0B / 1.71MB 0.2s
```

release-image / integrated-pipeline

succeeded 1 hour ago in 42s

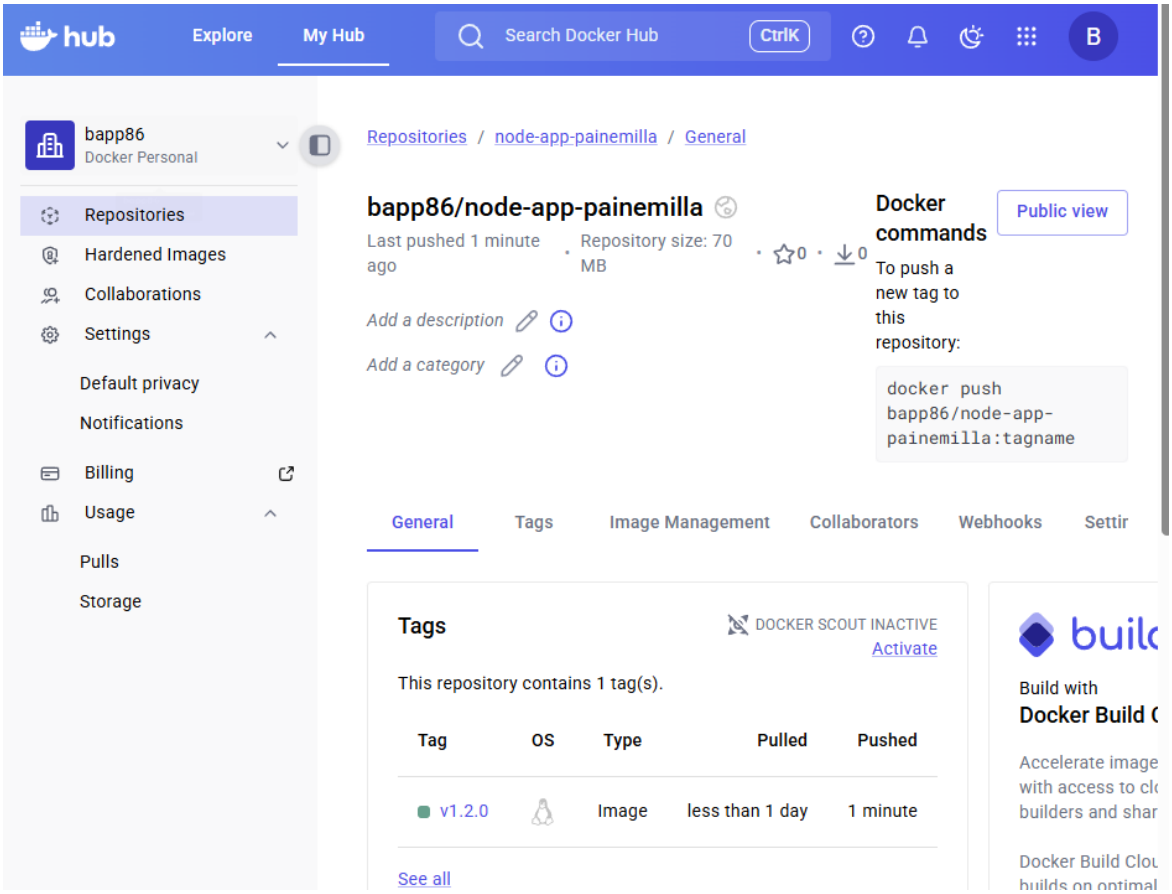
Search logs

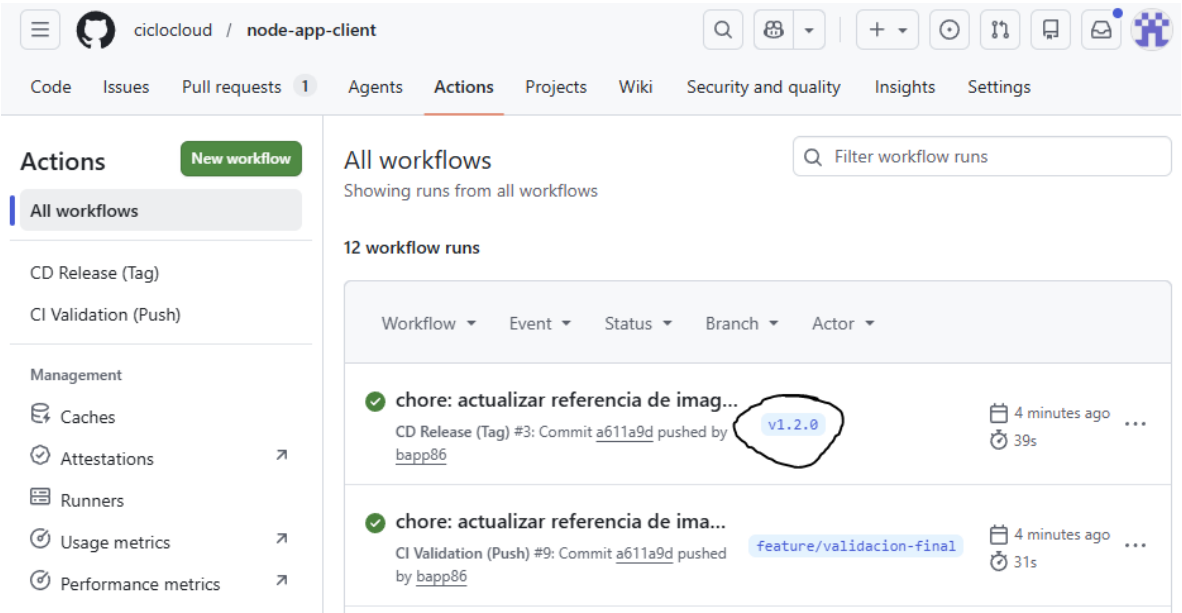
Build and Push 22s

```
sha256:d654216581d89a2b9a2ecd38c153c80b55e002651fd2e1f519ce4c90892b5457
#6 extracting
sha256:d654216581d89a2b9a2ecd38c153c80b55e002651fd2e1f519ce4c90892b5457 0.9s done
#6 DONE 3.1s

#6 [1/5] FROM docker.io/library/node:20-
slim@sha256:f93745c153377ee2fbbdd6e24efcd03cd2e86d6ab1d8aa9916a3790c40313a55
#6 extracting
sha256:9b97559caf99ea6450224af9cae9da7d2a54713725ed842cff9b4e47dfa6f3a2 0.0s done
#6 extracting
sha256:c229a5665a7712c6ffed00bc6c69aa9c0863da6c57230a2a49f67b65ad46cc60 done
#6 DONE 3.2s

#8 [2/5] WORKDIR /app
#8 DONE 0.3s

#9 [3/5] COPY package*.json ./
#9 DONE 0.0s

#10 [4/5] RUN npm ci --only=production
#10 0.131 npm warn config only Use `--omit=dev` to omit dev dependencies from the
install.
#10 1.015
#10 1.015 added 65 packages, and audited 66 packages in 910ms
#10 1.015
#10 1.015 22 packages are looking for funding
#10 1.015   run `npm fund` for details
#10 1.016
#10 1.016 found 0 vulnerabilities
#10 1.018 npm notice
#10 1.018 npm notice New major version of npm available! 10.8.2 -> 11.12.1
#10 1.018 npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.12.1
#10 1.018 npm notice To update run: npm install -g npm@11.12.1
#10 1.018 npm notice
#10 DONE 1.0s

#11 [5/5] COPY . .
#11 DONE 0.2s
```

5. Conclusiones

La arquitectura implementada para TechMaster LTDA no solo cumple con los requerimientos técnicos de desacoplamiento, sino que establece una base sólida para el escalamiento. El control de ejecución basado en eventos de Tag garantiza la trazabilidad y seguridad del software, mientras que la centralización de plantillas asegura que todas las futuras aplicaciones de la organización sigan los mismos estándares de calidad y rendimiento definidos en esta fase.





Confirmación final del despliegue exitoso en Docker Hub. Se observa el repositorio institucional 'node-app-painemilla' con el tag de versión v1.2.0 debidamente publicado. Esta evidencia cierra el ciclo de vida del software, demostrando que la arquitectura desacoplada entrega artefactos inmutables y listos para producción.